NATIONAL TECHNICAL UNIVERSITY OF UKRAINE
“IGOR SIKORSKY KYIV POLYTECHNIC INSTITUTE”

Software Engineering Components 4
Software Quality and Testing

LABORATORY WORK #1
Software Inspection and Analysis (Part 2)

Using Software Tools to Implement a Module for Static Software Analysis

Student	Xiaodi Wang
Group	KP-44i
Program	Bachelor in Software Engineering
Faculty	Faculty of Program Systems and Applied Mathematics
University	NTUU KPI
Computer OS	macOS

Kyiv - 2026

1. Purpose and Objectives

The purpose of this laboratory work is to study static software analysis and implement a small module that can inspect source code without executing it. The practical objectives are to detect code-quality, style, maintainability, and security problems; generate machine-readable and human-readable reports; and interpret the significance of the findings.

2. Tools and Environment

- Programming language: Python 3.
- Development environment: macOS; Terminal and a code editor/IDE.
- Static-analysis tools: Pylint for general code quality, Flake8 for style and syntax checks, and Bandit for security-oriented inspection.
- Additional implementation: a dependency-free AST/text fallback inspector included in static_analyzer.py.
- Repository account: https://github.com/XiaodiDennis.

3. Input Source Code

The file sample_code.py intentionally contains several minor and security-related defects. These defects are included so that the static-analysis tools have meaningful findings to report. The project also contains fixed_sample_code.py, which demonstrates remediation of the detected problems.

4. Static Analysis Module

The module static_analyzer.py accepts a Python file and an output directory. It attempts to execute Pylint, Flake8, and Bandit through subprocess without executing the analyzed program itself. It stores each tool output in a separate text file. It also performs a compact AST/text inspection and produces a JSON summary plus an HTML report.

Main command used on macOS:

```
python3 static_analyzer.py sample_code.py --output reports/original
```

Individual tool commands:

```
pylint sample_code.py
```

```
flake8 sample_code.py
```

```
bandit -r sample_code.py
```

5. Detected Issues

The built-in inspection identified 14 issues in the original sample. The table below summarizes the findings. Pylint, Flake8, and Bandit provide complementary external reports when the requirements are installed on the student computer.

Line	Severity	Code	Issue	Significance
3	WARNING	F401	Imported name	Unused imports

			'os' is unused	add noise and can indicate unfinished or unnecessary dependencies.
4	WARNING	F401	Imported name 'sys' is unused	Unused imports add noise and can indicate unfinished or unnecessary dependencies.
8	HIGH	S001	Hard-coded password-like string	Hard-coded credentials can be exposed through source control or distribution.
11	INFO	D001	Function 'CalculateTotal' has no docstring	Missing documentation makes maintenance and review more difficult.
11	WARNING	N001	Function name 'CalculateTotal' is not snake_case	Naming violations reduce consistency and readability.
13	WARNING	F841	Local variable appears intentionally unused	Unused variables may indicate dead code or incomplete logic.
17	WARNING	E501	Line too long (168 > 88 characters)	Very long lines reduce readability and complicate code review.
21	INFO	D001	Function 'choose_token' has no docstring	Missing documentation makes maintenance and review more difficult.
22	MEDIUM	S003	Pseudo-random generator used where security-sensitive use may require secrets	The random module is not suitable for security-sensitive token generation.
25	INFO	D001	Function 'run_command' has no docstring	Missing documentation makes maintenance and review more difficult.

26	HIGH	S002	subprocess.run(.. ., shell=True) may allow command injection	shell=True with constructed input can create command-injection risk.
29	WARNING	N002	Class name 'useraccount' is not PascalCase	Class naming should follow the project language conventions.
30	INFO	D001	Function '__init__' has no docstring	Missing documentation makes maintenance and review more difficult.
33	INFO	D001	Function 'display' has no docstring	Missing documentation makes maintenance and review more difficult.

6. Remediation

- Removed unused imports and the deliberately unused local variable.
- Renamed CalculateTotal to calculate_total and useraccount to UserAccount.
- Added docstrings and type annotations to improve maintainability.
- Shortened the long output line.
- Removed the hard-coded password-like constant.
- Replaced random.randint with the secrets module for token generation.
- Replaced shell=True command construction with a list of subprocess arguments and check=True.

The dependency-free fallback analysis reports 1 issues in fixed_sample_code.py, compared with 14 in the original sample. This provides a direct before/after verification of the remediation.

7. Generated Reports

- reports/original/pylint_report.txt - raw Pylint output.
- reports/original/flake8_report.txt - raw Flake8 output.
- reports/original/bandit_report.txt - raw Bandit security output.
- reports/original/builtin_analysis_report.txt - dependency-free inspection output.
- reports/original/summary.json - structured summary.
- reports/original/static_analysis_report.html - human-readable HTML summary.
- reports/fixed/ - the same report set for the corrected source code.

8. Interpretation

Static analysis is useful because it can reveal defects before runtime testing. Style findings such as naming, long lines, and missing documentation mostly affect readability and maintainability. Unused imports and variables can reveal unnecessary or unfinished code. Security findings require higher priority: hard-coded credentials can leak sensitive information, pseudo-random values are unsuitable for security tokens, and shell command construction can allow command injection if user-controlled input reaches the command string. Therefore, not every warning has the same operational risk, and findings should be classified before remediation.

9. Reproducibility Procedure on macOS

1. Open Terminal and change to /Users/hsiao/Downloads/KP44i_SQT_Lab1_Xiaodi_Wang.

2. Create a virtual environment: python3 -m venv .venv

3. Activate it: source .venv/bin/activate

4. Install tools: pip install -r requirements.txt

5. Run: ./run_analysis.sh

6. Open the HTML report: open reports/original/static_analysis_report.html

7. Confirm that reports/environment_and_versions.txt contains local tool versions before submission.

10. Conclusion

The laboratory work demonstrates a reproducible static-analysis workflow for Python. The implemented module inspects code without executing the target program, combines general quality, style, and security tools, stores raw outputs, and generates consolidated reports. The exercise shows that static analysis is most effective when findings are interpreted by severity and then verified after code correction.

Appendix A. Submission Package Structure

```
KP44i_SQT_Lab1_Xiaodi_Wang/
├── sample_code.py
├── fixed_sample_code.py
├── static_analyzer.py
├── requirements.txt
├── run_analysis.sh
├── README.md
├── reports/
│   ├── original/
│   ├── fixed/
│   └── environment_and_versions.txt
└── docs/
    ├── Lab_Report_Xiaodi_Wang.docx
    └── Lab_Report_Xiaodi_Wang.pdf
```